

Yolo (You only look Once): object detection algorithm

for object : Sliding window $\rightarrow$ RCNN $\rightarrow$ fast RCNN
detection Object Detection
(images) $\downarrow$
faster RCNN
 $\downarrow$
YOLO

Image classification: Neural network output: Dog=1, person=0

Object localisation: Neural network output:-

$\text{Dog}=1$
 $\text{person}=0$
+
Bounding box

we have defined
2 class here Dog &
person

(B_x, B_y)
 B_w

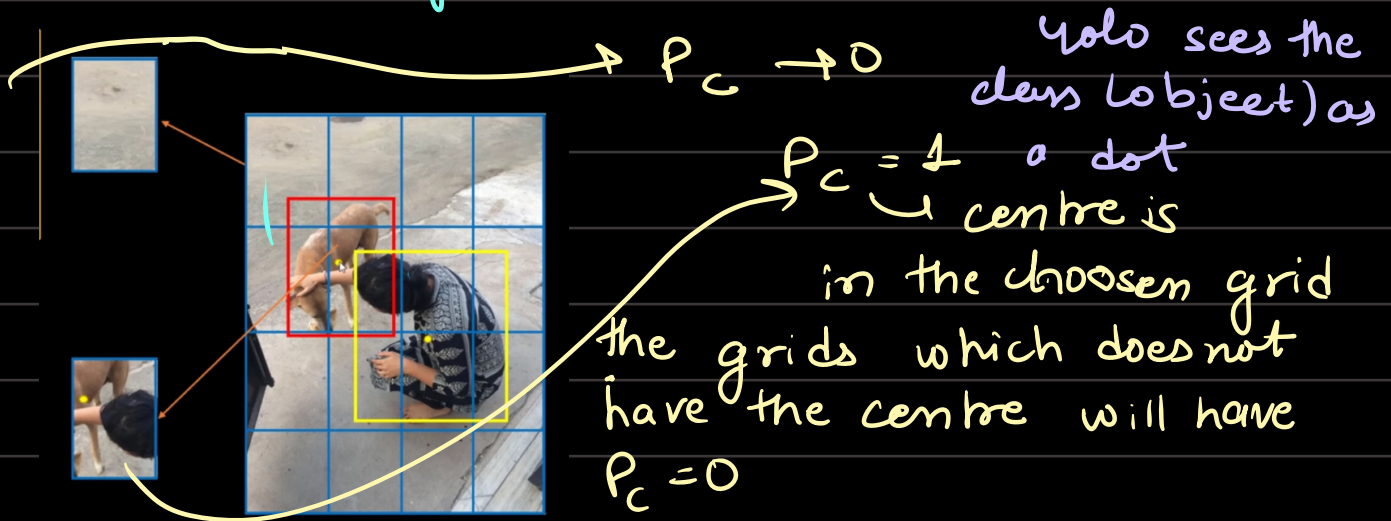
$\begin{bmatrix} P_c \\ B_x \\ B_y \\ B_w \\ B_h \\ C_1 \\ C_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 50 \\ 70 \\ 60 \\ 70 \\ 1 \\ 0 \end{bmatrix}$ vector
 P_c : probability for the image inside the box being

from the class
its a dog $\rightarrow 1$

$x_{\text{-train}}$ $y_{\text{-train}}$

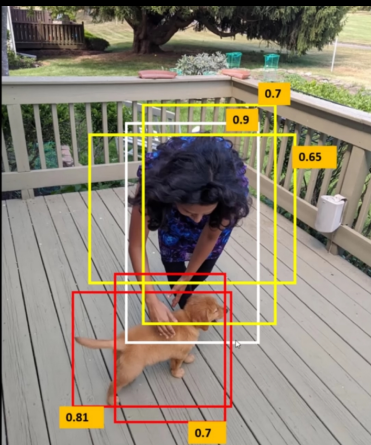
$x_{\text{-train}}, y_{\text{-train}} \rightarrow \text{CNN} \rightarrow \text{vector (final)}$

Case :- having multiple objects



→ 16 vectors

In this case when the image is passed to CNN → it will divide the image into 16 grids & thus finding the bounding box, But multiple bounding box can be detected?



→ this will be solved using IOU

IOU : intersection over union

⇒ intersection area

union area

The bounding box having the highest IOU will be the best predicted bounding box

Note: It may happen that a single grid contains both the centres of both the objects

→ just concatenate the two vectors

→ will consist of two anchor boxes

So, YOLO localizes object by dividing the image into grids (CNN), predicting many BB, evaluating IOU (training) & applying non-max suppression to output final bounding box.

Task-3

Repeat task-1 using YOLO (YOLO model & YOLOv11s.pt)

We are given a pretrained model, import it in google colab



Code:

```
!pip install ultralytics opencv-python -q
```

1.2/1.2 MB 13.4 MB/s eta 0:00:00

```
[20] from ultralytics import YOLO
import cv2
import numpy as np
from google.colab import files
from matplotlib import pyplot as plt
```

```
[3] uploaded = files.upload()
```

```
uploaded = files.upload()
```

Choose files YOLOv11s-Carmaker.pt
YOLOv11s-Carmaker.pt(n/a) - 19160979 bytes, last modified: 12/02/2026 - 100% done
Saving YOLOv11s-Carmaker.pt to YOLOv11s-Carmaker.pt

```
[4] uploaded = files.upload()
```

Choose files image.png
image.png(image/png) - 1779980 bytes, last modified: 12/02/2026 - 100% done
Saving image.png to image.png

→ importing the image

we have

to import this pretrained model of YOLOv11s-Carmaker otherwise the model will not identify the cones as class (objects)

```
[5] image_path = "image.png"
image = cv2.imread(image_path)
if image is None:
    print("Image not loaded - check filename!")
else:
    print("Image loaded successfully")
```

```
Image loaded successfully
```

```
Image loaded successfully

[7] 0s results = model(image_path)

***
image 1/1 /content/image.png: 384x640 3 Blues, 2 Small Oranges, 3 Yellows, 487.4ms
Speed: 12.0ms preprocess, 487.4ms inference, 36.9ms postprocess per image at shape (1, 3, 384, 640)
```

```
[46] 0s img_with_boxes = image.copy()
f = 1000
H = 25

for box in results[0].boxes:
    x1, y1, x2, y2 = map(int, box.xyxy[0])
    h = y2 - y1
    if(h==0):
        continue
    depth = (f * H) / h
    class_id = int(box.cls)
    if(class_id == 1):
        c = (255, 0, 0)
    elif(class_id == 2):
        c = (0, 140, 255)
    else:
        c = (0, 255, 255)
    cv2.rectangle(img_with_boxes, (x1,y1), (x2,y2), c, 2)
    depth_text = f"{depth:.2f}"
    cv2.putText(img_with_boxes, depth_text, (x1, y1-10),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5, c, 2)
```

→ yolov11s - carmaker.pt
returns (x1,y1) & (x2,y2)
& not (xc,yc)

[47]
4s

```
img_rgb = cv2.cvtColor(img_with_boxes, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(8,6))
plt.imshow(img_rgb)
plt.axis("off")
```

```
(np.float64(-0.5), np.float64(1279.5), np.float64(719.5), np.float64(-0.5))
```

